

- 资配订单匹配问题
 - 一、背景
 - 二、当前问题
 - 二、问题建模
 - 三、基于匈牙利算法的经典方案
 - 1. 数学建模
 - 2. 算法的困难点：
 - 3. 解法：
 - 四、基于KM算法的经典方案
 - 1. 数学建模
 - 2. 算法的困难点：
 - 3. 解法（思路）：
 - 五、方案三：推荐模型方案
 - 1. 数学建模
 - 2. 算法的难点：
 - 3. 解法（思路）：
 - 六、实验计划
 - 1. 仿真系统构建
 - 2. 算法评估指标（即业务关心的一系列优化目标）
 - 3. 加入对比的算法方案
 - 4. A/B测试及全量推流
 - 5. 在灰度实验上有显著效果后可全量推流

资配订单匹配问题

一、背景

由于外部政策收紧，资金规模持续缩减，出现资产大于资金的情况。通俗的来讲，就是客户的借款需求大于钱不够分了，需要给资方找到优质的客户。

类似的场景有很多，这些都属于二分图匹配问题的典型应用：

- **出行领域**：滴滴/货拉拉的司乘匹配，美团外卖的骑手订单匹配 [滴滴司乘匹配论文](#)
- **电商领域**：淘宝/京东的商品推荐，拼多多的团购匹配
- **金融领域**：同业的资金资产匹配，助贷平台的借贷匹配

对于此类二分图匹配问题，核心都是在两个不同集合之间建立最优的一对一或一对多映射关系，通过优化目标函数来实现资源的最优配置。

针对资配订单匹配这一具体场景，最常见的解决方案有两个：**二分图最优化**和**推荐算法**。**二分图最优化**（如匈牙利算法/KM算法）侧重于全局最优解，通过数学建模确保匹配结果在理论上达到最优；缺点就是实时性稍逊一筹。**推荐算法**则更注重当前订单的个性化匹配，通过机器学习模型学习历史数据中的复杂模式，在处理大规模数据时具有更好的可扩展性和实时性；缺点则是学习全局最优的目标较难。

二、当前问题

当前资配业务面临三大核心挑战：

- 资产资金结构性失衡**：政策收紧导致资金规模缩减，出现资产大于资金的供需矛盾，同时存在区域资金利用不均衡问题（全国资金放完但区域资金未满足）
- 用户体验与资产质量双重压力**：查征次数过多导致用户投诉，路由长度过长影响用户体验，优质用户资产利用率不足造成客户流失
- 匹配效率有待提升**：当前基于“首推模型TOP3+兜底资方”的策略在处理积压订单（占比30%+）时效果有限，需要更精细化的运筹优化方案

二、问题建模

常用术语：

- $F = \{f_1, f_2, \dots, f_m\}$ ：资方集合，是当前和嘉银进行合作的资方列表
- $F_{\text{区域}} = \{f_{r1}, f_{r2}, \dots, f_{rk}\} \subseteq F$ ：区域资方集合，是具有区域资金限制的资方子集
- $O = \{o_1, o_2, \dots, o_n\}$ ：订单集合，是当前需要考虑进行分配的订单列表，可能是一天内的所有订单
- $O_{\text{优质}} = \{o_{q1}, o_{q2}, \dots, o_{ql}\} \subseteq O$ ：优质订单集合，是根据风险评级、客户价值等指标筛选出的优质订单子集
- $A = \{a_1, a_2, \dots, a_n\}$ ：订单金额，其中 a_o 表示订单 o 的金额
- $C = \{c_1, c_2, \dots, c_m\}$ ：资方资金容量，其中 c_f 表示资方 f 的可用资金额度
- $C_R = \{c_{r1}, c_{r2}, \dots, c_{rk}\}$ ：区域资方资金容量，其中 c_{rf} 表示区域资方 f_r 的可用区域资金额度
- $Adj \in \{0, 1\}^{n \times m}$ ：可行性邻接矩阵，其中 $Adj_{o,f} = 1$ 表示订单 o 可以分配给资方 f ， $Adj_{o,f} = 0$ 表示不可行
- $P \in [0, 1]^{n \times m}$ ：通过率矩阵，其中 $P_{o,f}$ 表示订单 o 分配给资方 f 的授用信通过率

- $X \in \{0, 1\}^{n \times m}$: 匹配决策变量, 其中 $X_{o,f} = 1$ 表示订单 o 分配给资方 f , $X_{o,f} = 0$ 表示不分配

优化目标:

在满足资方容量约束和可行性约束的前提下, 找到最优匹配策略 X^* 。根据不同的业务目标, 有着不同的优化方向:

- **区域资金利用率最大化**: 最大化区域资方实际放款金额与区域资方总可用资金的比值

$$X^* = \arg \max_X \frac{\sum_{o \in O} \sum_{f \in F_{\text{区域}}} X_{o,f} \cdot a_o \cdot P_{o,f}}{\sum_{f \in F_{\text{区域}}} C_f}$$

- **优质资产承接率最大化**: 最大化优质资产的成功匹配比例

$$X^* = \arg \max_X \frac{\sum_{o \in O_{\text{优质}}} \sum_{f \in F} X_{o,f} \cdot a_o \cdot P_{o,f}}{\sum_{o \in O_{\text{优质}}} a_o}$$

- **通过率最大化**: 最大化订单通过数与订单总数的比值
- **盈利金额最大化**: 最大化总盈利金额
- **资金利用率最大化**: 最大化实际放款金额与资方总可用资金的比值

$$X^* = \arg \max_X \frac{\sum_{o \in O} \sum_{f \in F} X_{o,f} \cdot a_o \cdot P_{o,f}}{\sum_{f \in F} C_f}$$

约束条件:

- $\sum_{f \in F} X_{o,f} \leq 1, \forall o \in O$: 每个订单最多分配给一个资方
- $\sum_{o \in O} X_{o,f} \cdot a_o \leq c_f, \forall f \in F$: 资方资金计划限制
- $\sum_{o \in O} X_{o,f} \cdot a_o \leq c_{rf}, \forall f \in F_{\text{区域}}$: 区域资方资金计划限制
- $X_{o,f} \leq Adj_{o,f}, \forall o, f$: 可行性约束 (挡板)
- $\frac{\sum_{o \in O} X_{o,f} \cdot a_o \cdot R_o}{\sum_{o \in O} X_{o,f} \cdot a_o} \leq R_f^{\max}, \forall f \in F$: 资方风险要求约束, 其中 R_o 表示订单 o 的风险率 (如A级3%、B级5%、C级7%、D级9%), $R_f^{\max}$ 表示资方 f 可接受的最大加权平均风险率
- $X_{o,f} \in \{0, 1\}, \forall o, f$: 决策变量约束 (订单必须完整的被分配)

三、基于匈牙利算法的经典方案

1. 数学建模

目标函数（以资金利用率最大化举例）：

$$\max \frac{\sum_{i=1}^n \sum_{j=1}^m X_{i,j} \cdot a_i \cdot Adj_{i,j}}{\sum_{j=1}^m c_j}$$

约束条件：

- $\sum_{j=1}^m X_{i,j} \leq 1, \forall i \in \{1, \dots, n\}$ ：每个订单最多分配给一个资方
- $\sum_{i=1}^n X_{i,j} \cdot a_i \leq c_j, \forall j \in \{1, \dots, m\}$ ：资方资金计划限制
- $X_{i,j} \leq Adj_{i,j}, \forall i, j$ ：可行性约束（挡板），其中 $Adj_{i,j} = 1$ 表示订单 o_i 和资方 f_j 之间存在可行的匹配边
- $X_{i,j} \in \{0, 1\}, \forall i, j$ ：决策变量约束（订单必须完整的被分配）

2. 算法的困难点：

- **边存在性预测困难**：匈牙利算法要求准确预测每条边是否存在（即 $Adj_{i,j}$ 的值），这需要精确判断订单 o_i 是否能够被资方 f_j 接受。在实际业务中，这种预测较难，因为涉及复杂的资方风控策略和动态的市场条件。
- **无法处理带权重的优化**：标准匈牙利算法主要解决最大匹配问题，而资配场景更关注带权重的优化目标（如资金利用率、通过率等），需要转换为最大权重匹配问题。

3. 解法：

四、基于KM算法的经典方案

1. 数学建模

目标函数（以资金利用率最大化举例）：

$$\max \frac{\sum_{i=1}^n \sum_{j=1}^m X_{i,j} \cdot a_i \cdot P_{i,j}}{\sum_{j=1}^m c_j}$$

约束条件：

- $\sum_{j=1}^m X_{ij} \leq 1, \forall i \in \{1, \dots, n\}$: 每个订单最多分配给一个资方
- $\sum_{i=1}^n X_{ij} \cdot a_i \leq c_j, \forall j \in \{1, \dots, m\}$: 资方资金计划限制
- $X_{ij} \leq Adj_{ij}, \forall i, j$: 可行性约束 (挡板), 其中 $Adj_{ij} = 1$ 表示订单 o_i 和资方 f_j 之间存在可行的匹配边
- $X_{ij} \in \{0, 1\}, \forall i, j$: 决策变量约束 (订单必须完整的被分配)

2. 算法的困难点:

- **通过率预测精度要求高**: KM算法需要精准的边权重 (通过率预测), 否者会影响最优化的方向。但现有的按业务规则打包的方式, 客群内订单方差较大 (人的特征各式各样)。需要精准的对客户进行分群, 估计出特定客群的通过率, 这在实际业务中具有很大挑战性。
- **批处理延迟问题**: 全局最优需要完整订单集合, 而实际订单是陆陆续续来的, 资源约束需要统筹协调, 订单间存在竞争关系。

3. 解法 (思路) :

通过率预测精度要求高 - 基于聚类获取稳定的客群通过率: 基于订单特征, 如 $\mathbf{o}_i = [\text{金额}_i, \text{期限}_i, \text{风险}_i, \text{行业}_i, \text{地区}_i]$ 等, 进行聚类, 将相似订单归为同一类别:

$$C_k = \{o_i \mid \arg \min_k \|\mathbf{f}(o_i) - \mu_k\|^2\}$$

其中 $\mathbf{f}(o_i)$ 为订单 o_i 的特征向量, μ_k 为第 k 个聚类中心。

通过率估计: 其中 $P_{i,j}$ 通过考虑人维度的各种标签进行聚类, 稳定估计通过率:

$$P_{i,j} = P(\text{通过} \mid o_i \in C_{\text{cluster}(i)}, f_j) = \frac{\sum_{o \in C_{\text{cluster}(i)}} \text{历史通过数}_{o,j}}{\sum_{o \in C_{\text{cluster}(i)}} \text{历史总数}_{o,j}}$$

聚类对KM算法的关键作用:

- **数据稳定性**: 通过聚类获得足够相似的样本量来稳定估计通过率
- **业务合理性**: 确保相似订单获得相似的匹配策略

批处理延迟问题 - 最优-等待平衡, 及资金划分策略:

1. **时间窗口**: 固定时间窗口 (如10分钟) 触发批处理
2. **数量阈值**: 订单数达到阈值时触发
3. **混合触发**: 时间或数量任一条件满足即触发

4. **优先级分层**：紧急订单（如大权益/权益加速订单）贪心算法处理，普通订单批处理
5. **资金计划划分**：如果采用1小时一次的批处理优化，需要对一天的资金量进行合理划分，避免早期批次消耗过多资金导致后续批次无资金可用
6. **时段权重分配**：基于历史订单分布规律，将每日资金按时段权重分配给各个批处理窗口，确保全天资源配置的均衡性

五、方案三：推荐模型方案

1. 数学建模

推理阶段目标函数（以通过率最大化为例）：

$$\max \sum_{i=1}^n \sum_{k=1}^m x_{i,k} \cdot \hat{P}(\text{通过} \mid o_i, f_k, \theta^*)$$

其中 θ^* 为训练得到的最优参数。

2. 算法的难点：

- **目标函数与特征建模复杂**：推荐系统通常针对单一目标优化，而资配场景需同时考虑资金利用率、通过率、盈利金额等多个业务目标。同时，订单通过与否不仅取决于静态特征，还受市场环境、资方实时状态等动态因素影响，导致模型训练困难。

3. 解法（思路）：

目标函数与特征建模复杂 - 多目标优化解决方案：方案一：基于综合评分的单一决策模型 由于资金利用率和盈利金额等目标在订单间存在竞争关系，传统的多任务学习框架难以有效建模。因此采用综合评分的方式，将多个业务目标融合为单一决策分数：

$$Score(o_i, f_k) = \alpha_1 \cdot \hat{P}(\text{通过} \mid o_i, f_k) + \alpha_2 \cdot \frac{a_i \cdot \hat{P}(\text{通过} \mid o_i, f_k)}{c_k} + \alpha_3 \cdot profit_{i,k}$$

其中：

- $\hat{P}(\text{通过} \mid o_i, f_k)$ ：订单 o_i 在资方 f_k 的通过率预测
- $profit_{i,k}$ ：订单 o_i 匹配资方 f_k 的预期盈利

- $\alpha_1, \alpha_2, \alpha_3$ 为权重系数，通过业务重要性或离线验证效果调节

模型训练目标：

$$L = - \sum_{i,k} y_{i,k} \log \sigma(\text{Score}(o_i, f_k)) + (1 - y_{i,k}) \log(1 - \sigma(\text{Score}(o_i, f_k)))$$

其中 $y_{i,k} = 1$ 表示订单 o_i 历史上成功匹配给资方 f_k 且通过， $y_{i,k} = 0$ 表示未匹配或匹配后被拒绝。

本方案的优势在于：

1. **全局一致性**：单一评分函数确保决策的一致性
2. **业务目标平衡**：通过权重系数灵活平衡不同业务目标
3. **订单竞争建模**：可以在后续的全局优化阶段考虑订单间的资源竞争关系

方案二：多目标模型排序融合 (Rank Fusion) 分别训练单目标模型，然后通过排序融合获得最终匹配结果：

单目标模型训练：

- $M_1(o_i, f_k) \rightarrow \hat{P}_{\text{通过}}$ - 通过率模型：需要训练模型预测订单在各资方的通过概率
- $M_2(o_i, f_k) \rightarrow \text{Score}_{\text{经验}}$ - 人工经验模型：基于业务专家经验规则的评分，如资方偏好、风险等级匹配等
- $\text{profit}_{i,k} = a_i \times \text{rate}_k$ - 盈利金额排序：按资方利率从高到低排序，无需训练模型

排序融合策略： 对于订单 o_i ，各模型给出资方排序列表：

- $\text{Rank}_1(o_i) = [f_{j_1}, f_{j_2}, \dots, f_{j_m}]$ (按通过率排序)
- $\text{Rank}_2(o_i) = [f_{k_1}, f_{k_2}, \dots, f_{k_m}]$ (按资金利用率排序)
- $\text{Rank}_3(o_i) = [f_{l_1}, f_{l_2}, \dots, f_{l_m}]$ (按盈利金额排序)

Borda Count融合：

$$\text{Score}(o_i, f_j) = \beta_1 \cdot (m - \text{rank}_1(f_j)) + \beta_2 \cdot (m - \text{rank}_2(f_j)) + \beta_3 \cdot (m - \text{rank}_3(f_j))$$

其中 $\text{rank}_k(f_j)$ 表示资方 f_j 在第 k 个排序列表中的位置， $\beta_1, \beta_2, \beta_3$ 为融合权重。

本方案的优势在于：

1. **单一目标建模**：每个子模型专注于单一业务目标，数据更好获取，算法更稳定
2. **模块化设计**：各子模型可独立训练和优化，便于维护和迭代

3. **可解释性强**：每个排序结果都有明确的业务含义，便于业务理解和调试

六、实验计划

1. 仿真系统构建

仿真系统架构：

- **核心设计理念**：构建高保真度的离线仿真环境，支持多种算法方案的安全对比实验，避免直接上线对业务造成冲击
- **资方状态模拟器**：实时模拟资方资金状态、风控策略变化和决策延迟，确保仿真环境的真实性
- **多算法同场竞技**：支持多个配资算法在相同数据集上并行运行，确保公平对比
- **仿真精度控制**：基本还原历史订单路由环境，仿真精度>95%，通过历史回放验证算法准确性
- **全面指标输出体系**：自动输出资金利用率、通过率、利润率、新客承接率、优质资产承接率等关键业务指标

仿真能力边界：

-  支持：算法逻辑验证、指标对比、参数调优
-  不支持：实时资方反馈、外部环境变化影响
-  近似模拟：资方决策延迟、系统负载影响

仿真系统的验证指标： $\text{历史回放准确性} = \frac{\text{仿真结果与历史实际一致的订单数}}{\text{历史总订单数}}$

2. 算法评估指标（即业务关心的一系列优化目标）

仿真系统输出各项指标评估不同的算法效果：

$$\begin{aligned}
 \text{各算法效果评估指标} = \left\{ \begin{aligned}
 &\text{通过率} = \frac{\sum \text{订单通过数}}{\sum \text{订单总数}} \\
 &\text{资金利用率} = \frac{\text{实际放款金额}}{\text{资方总可用资金}} \\
 &\text{资产利用率} = \frac{\text{成功匹配资产数}}{\text{总可匹配资产数}} \\
 &\text{优质资产承接率} = \frac{\text{优质资产成功匹配数}}{\text{优质资产总数}} \\
 &\text{新客率} = \frac{\text{新客户成功匹配数}}{\text{总成功匹配数}} \\
 &\text{盈利金额} = \sum_{i=1}^n (\text{放款金额}_i \times \text{利率}_i) \\
 &\text{平均路由长度} = \frac{\sum_{i=1}^n \text{路由步数}_i}{n}
 \end{aligned} \right.
 \end{aligned}$$

3. 加入对比的算法方案

对比方案：

- 方案一：现有规则引擎的贪心算法（基准）
- 方案二：现有的资配首推模型
- 方案三：聚类+KM算法
- 方案四：深度学习推荐算法

在仿真系统上有显著效果后可进行A/B实验上线验证。

4. A/B测试及全量推流

灰度策略：

- 第一阶段：5%流量（A组2.5% vs B组2.5%），观察2周
- 第二阶段：20%流量（A组10% vs B组10%），观察2周

关键指标：资金利用率、通过率、匹配成功率、响应时间、系统稳定性

5. 在灰度实验上有显著效果后可全量推流